

- A lógica bit a bit tem o objetivo de alterar apenas um bit e manipular as GPIOs (entradas e saídas)
- Controle individual de bits
- Esse controle acontece devido ao uso de operadores

Operador	Nome	Uso principal
	OU lógico bit a bit	Ativar (colocar em 1) bits específicos
&	E lógico bit a bit	Limpar (colocar em 0) bits específicos
^	OU Exclusivo (XOR)	Inverter/alternar o estado de bits
~	Complemento de 1 (NOT)	Inverter todos os bits
<<	Deslocamento à esquerda	Mover bits para a esquerda
>>	Deslocamento à direita	Mover bits para a direita

Operação & (AND)

- O resultado é 1 somente quando os dois bits são 1

Tabela verdade do AND:

```
0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1 ← único caso que resulta em 1
```

- Uso prático: O AND é usado para ler/testar bits específicos (máscara) e para apagar bits (zerar)

Operação | (OU)

- resultado é 1 quando pelo menos um dos bits é 1 só resulta em 0 quando os dois são 0

Tabela verdade do OR:

```
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1
```

- Uso prático: O OR é usado para ativar (setar) bits específicos sem alterar os demais

Operação ~ (NOT)

- Inverte todos os bits de uma vez o que era 0 vira 1, e o que era 1 vira 0
- Raramente usado sozinho, usado muito em conjunto com &~ para desligar bits individualmente

```
0 = 1
1 = 0
```

Operação ^ (XOR)

- o resultado é 1 quando os bits são diferentes entre si

```
Tabela verdade do XOR:
0 ^ 0 = 0    (iguais → 0)
0 ^ 1 = 1    (diferentes → 1)
1 ^ 0 = 1    (diferentes → 1)
1 ^ 1 = 0    (iguais → 0)
```

- O XOR é perfeito para inverter (toggle) bits específicos ligar o que estava desligado e vice-versa

Deslocamento à Esquerda <<

- Move todos os bits para a esquerda, e os bits que saem pela esquerda são perdidos, zeros entram pela direita

```
y = 1010
x = y << 1 → x = 0100
```

```
Antes:  1 0 1 0
        ✓ ✓ ✓ ✓
```

```
Depois: 0 1 0 0 (o 1 da esquerda saiu, um 0 entrou pela direita) (todos se movem pra esquerda)
```

- $1 \ll N$ gera uma máscara com apenas o bit N ativado. Por exemplo, $1 \ll 4 = 00010000$ isso é muito usado para indicar qual bit você quer manipular

Deslocamento à Direita >>

- Move todos os bits para a direita, zeros entram pela esquerda

```
y = 1010
x = y >> 1 → x = 0101
```

Antes: 1 0 1 0

↘ ↘ ↘ ↘

Depois: 0 1 0 1 (um 0 entrou pela esquerda, o 0 da direita saiu)

As 3 operações essenciais

Ligar um bit:

```
status = status | (1 << 4); // Liga o bit 4
```

- Como funciona passo a passo:

status atual:	1 0 1 0 0 0 1 1	(valor original, preservar tudo)
máscara:	0 0 0 1 0 0 0 0	(1 << 4 = bit 4 ativado)
resultado OR:	1 0 1 1 0 0 1 1	← bit 4 agora é 1, resto intacto!

Desligar um bit:

```
status = status &~ (1 << 4); // Desliga o bit 4
```

- Entendendo em duas etapas:

Etapa 1: O NOT ~ inverte a máscara:

máscara original:	0 0 0 1 0 0 0 0	(1 << 4)
~máscara:	1 1 1 0 1 1 1 1	(todos os bits invertidos)

Etapa 2: O AND com a máscara invertida:

status atual:	1 0 1 1 0 0 1 1
~máscara:	1 1 1 0 1 1 1 1
resultado AND:	1 0 1 0 0 0 1 1 ← bit 4 agora é 0, resto intacto!

Testar um bit

```
if (status & (1 << 2))    // Testa se o bit 2 está em 1
```

```
status:  1 0 1 1 0 1 0 1
```

```
máscara: 0 0 0 0 0 1 0 0    (1 << 2)
```

```
resultado: 0 0 0 0 0 1 0 0    → não é zero → bit 2 está em 1!
```

- Se o resultado for diferente de zero → o bit testado está em 1. Se for zero → o bit está em 0

Resumo visual das 3 operações fundamentais:

OPERAÇÃO	CÓDIGO	EFEITO
LIGAR bit N	<code>x = x (1 << N)</code>	bit N → 1
DESLIGAR bit N	<code>x = x & ~(1 << N)</code>	bit N → 0
INVERTER bit N	<code>x = x ^ (1 << N)</code>	bit N troca
TESTAR bit N	<code>if(x & (1 << N))</code>	verifica se = 1